1. Normal Methods (Instance Methods)

These are the most common type of methods in Python classes.

- Take `self` as the **first parameter**.
- Can access **instance variables and methods**.
- Can access **class variables** using `self` or `ClassName`.

Example:

```
class MyClass:
    def __init__(self, value):
        self.value = value

    def show(self):  # normal method
        print("Value is:", self.value)

obj = MyClass(10)
obj.show()
```

2. Static Methods

These methods are not tied to a class instance.

- **Do not take `self` or `cls`** as the first argument.
- **Cannot access instance or class variables** directly.
- Use `@staticmethod` decorator to define.

◈ When to use:

Use static methods when you want a function that belongs to a class **logically**, but **doesn't need** to access or modify class/instance data.

Example:

```
class Math:
    @staticmethod
    def add(x, y):  # static method
        return x + y

print(Math.add(5, 3))  # no need to create object
```

Instance Variables vs Class Variables

1. Instance Variables

- Defined **inside the constructor** (`__init__`) using `self`.
- **Unique for each object**.
- Belong to **instances** (objects), **not** the class itself.

Example:

```
class Student:
  def __init__(self, name, marks):
```

```
    self.name = name      # instance variable
    self.marks = marks    # instance variable

s1 = Student("Neeraj", 90)
s2 = Student("Aman", 85)

print(s1.name) # Neeraj
print(s2.name) # Aman
```

2. Class Variables

- Defined **outside** all methods but **inside** the class.
- **Shared by all objects** of the class.
- Belong to the **class**, not to individual objects.
- You can access them using either ClassName.var or self.var (but modifying via self creates a new instance variable).

Example:

```
class Student:
  school = "Tech Mech Institute" # class variable

  def __init__(self, name):
    self.name = name      # instance variable

s1 = Student("Neeraj")
s2 = Student("Aman")

print(s1.school) # Tech Mech Institute
print(s2.school) # Tech Mech Institute

Student.school = "New School" # changing class variable
print(s1.school) # New School
```

Summary Table:

Feature	Instance Variable	Class Variable
Defined in	Constructor (`__init__`)	Inside class, outside methods
Belongs to	Object (instance)	Class (shared by all instances)
Accessed via	`self.variable`	`ClassName.variable` or `self.variable`
Storage	Unique copy per object	Single copy shared by class
Used for	Object-specific data	Common data for all objects

What is a Class Method?

- A class method is a method that works with the class, not the instance.
- It takes cls as the first argument instead of self.
- It is used to access or modify class variables.

Example:

```
class Student:
    school_name = "Tech Mech Institute"

    def __init__(self, name):
        self.name = name

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name

    def show(self):
        print(f"Name: {self.name}, School: {Student.school_name}")


# Before changing school
s1 = Student("Neeraj")
s1.show()  # Tech Mech Institute

# Change class variable using class method
Student.change_school("Future Tech School")
s1.show()  # Future Tech School
```

When to use `@classmethod`?

- You want to access or modify class variables.
- You are writing factory methods that return instances of the class.

What Is an Alternative Constructor?

An **alternative constructor** is a class method that returns an instance of the class using **custom logic** (like parsing a string, reading from a file, or processing data) instead of calling `ClassName(...)` with normal arguments.

Example: Using @classmethod as an Alternative Constructor

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data_str):
        name, age = data_str.split('-')
        return cls(name, int(age))

# Creating object normally
p1 = Person("Neeraj", 23)

# Creating object using alternative constructor
p2 = Person.from_string("Neeraj-23")

print(p1.name, p1.age)  # Neeraj 23
print(p2.name, p2.age)  # Neeraj 23
```

Why Use Class Methods as Alternative Constructors?

Benefit	Description
Readability	Easy to understand how object is created from different formats
Flexibility	You can have multiple ways to construct an object
Encapsulation	Keep initialization logic inside the class

Key Point:

- `@classmethod` receives the **class (`cls`)** as the first argument, not the instance (`self`).
- It can **return a new object** by calling `cls(...)`, i.e., the main constructor.

1. `dir()` – Introspection Function

Returns a list of **attributes and methods** associated with an object.

Example:

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Neeraj")
print(dir(p))
```

Output (partial):

['__class__', '__delattr__', '__dict__', 'name', '__init__', ...]

Key Points:

- Returns both user-defined and special methods.
- If used without arguments, it shows names in the current local scope.
- Useful for exploration/debugging.

2. __dict__ – Instance Attribute Dictionary

Returns a dictionary containing all **instance variables** (i.e., object's attributes).

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s = Student("Neeraj", 23)
print(s.__dict__)
```

Output:

```
{'name': 'Neeraj', 'age': 23}
```

Key Points:

- Shows only **data attributes** of the object, not methods.
- Returns a dictionary, so you can access, update, or inspect attributes.
- Commonly used in debugging and serialization (like JSON conversion).

---

3. help() – Documentation Function

Displays the **help/documentation** for a module, class, method, or function.

Example:

```
help(str.upper)
```

Output:

```
Help on built-in function upper:
upper(...)
    Return a copy of the string converted to uppercase.
```

Key Points:

- Shows what a function/class does.
- Very useful to understand unfamiliar functions/modules.
- Best used in interactive environments like Python shell or Jupyter Notebook.